Enterprise API Communication Gateway

In cloud-native applications, robust interaction with external RESTful services is a fundamental requirement. A production-grade communication gateway must manage HTTP method semantics, parse complex JSON payloads, and provide standardized error handling to transform raw network responses into actionable application data.

Implement an `APIHandler` class that encapsulates the logic for secure data retrieval and submission. The gateway must implement custom `APIError` reporting, handle diverse status codes (200, 201, 404, 500), and provide specialized methods for extracting high-value fields like user contact information while ensuring system stability during network failures.

Example 1 Input:

```
MOCK /users/1 200 {"id": 1, "email": "alice@api.com"} FETCH /users/1
```

Output:

```
{"email": "alice@api.com", "id": 1}
```

Explanation: The gateway executes a simulated GET request to the `/users/1` endpoint. Upon receiving a 200 OK status, it parses the JSON response and returns the user's data as a dictionary.

Example 2 Input:

```
USER_EMAIL 2
```

Output:

```
User Not Found
```

Explanation: The high-level `get_user_email` method attempts to resolve a user ID that does not exist (simulated 404). The gateway catches the resulting `APIError` and returns a user-friendly error message instead of crashing.

Function Parameter:

- endpoint (str): The relative API path.
- payload (dict): Data for POST submissions.
- user_id (int): Identifier for specific user lookups.

Returns

- dict/str: The parsed response body or a status identifier (e.g., "Created").

Constraints

- Raise a custom `APIError(status, message)` for all non-success codes.
- Success codes: 200 for GET, 201 for POST.
- The `get_user_email` method must be resilient to all network-level exceptions.

Input Format for Custom Testing Input from stdin contains commands: `MOCK <endpoint> <status> <body>`, `FETCH <endpoint>`, `SUBMIT <endpoint> <payload>`, or `USER_EMAIL <id>`.

Sample Case 0 Sample Input

```
MOCK /ping 200 {"status": "ok"} FETCH /ping
```

Sample Output

```
{"status": "ok"}
```

Explanation A simple connectivity check to the API endpoint returns a successful status dictionary.